

“My Credentials, Please”: Detecting Filesystem Capability Laundering in MCP Servers at Scale

Zeqian Huang¹, Mingyue Tang², Aonan Guan³, Qiming Sun⁴, and Pengzhou He⁵

¹ Independent Researcher

leilahuang95@gmail.com

² University of Illinois Urbana-Champaign, Urbana, IL, USA

mt55@illinois.edu

³ Independent Researcher

guanaonan@gmail.com

⁴ Santa Clara University, Santa Clara, CA, USA

qsun4@scu.edu

⁵ Auburn University at Montgomery, Montgomery, AL, USA

phe@aum.edu

Abstract. The Model Context Protocol (MCP) standardizes how AI agents invoke external tools. MCP servers expose tools through a declared contract consisting of a name, description, and argument schema. In typical local deployments, client-side approval of a tool call grants the underlying server process broad host filesystem permissions rather than restricting it to the declared operation. We study the resulting vulnerability class, *filesystem capability laundering*, where attacker-steered arguments cause a tool to access sensitive host files outside its declared contract and deployment scope. We analyze this capability gap across three layers: server-category screening (L1), per-tool contract analysis (L2), and bypassable path-validation checks (L3). We design MCP-FSD to catalog MCP servers, infer least-privilege boundaries per tool, plant credential fixtures outside those boundaries, and evaluate exploitability and sandbox containment via paired dynamic execution. Across 21,097 MCP servers from nine registries, 787 of 4,806 triaged servers (16.4%) exhibit filesystem capabilities despite non-filesystem functional classifications. Dynamic three-case validation confirms 150 vulnerable instances under tested sandbox policies. Coordinated disclosure yielded 13 security advisories, including two CVEs in Anthropic’s Git MCP Server. R3a, R3b

Keywords: Model Context Protocol · Filesystem Security · Sandbox · Static Analysis · AI Agent Security · Dynamic Validation · Tool-Use Security

1 Introduction

The Model Context Protocol (MCP) [17] has become the standard interface through which AI agents invoke external tools, data sources, and services. An MCP server implements tools that a client discovers through `tools/list` and

invokes through `tools/call`. The tool’s name, description, and argument schema form its *declared contract* [28].

MCP supports client-provided filesystem roots [27], and tool definitions can include output schemas and behavioral annotations [28]. However, the MCP specification delegates per-tool filesystem enforcement entirely to individual server implementations or host environments. The security question is whether attacker-influenced arguments can steer a tool’s effects beyond the scope justified by its contract and deployment context.

Path-traversal flaws are documented in the agent harness itself: Claude Code’s prefix-match bypass [5] and symlink-following [4] let crafted arguments escape the harness’s intended scope. For locally launched servers, the code that accesses files instead runs in a separate server process. While the client’s approval prompt gates execution based on declared arguments, the spawned server process executes with ambient host filesystem permissions. An ostensibly benign operation can therefore access and expose host credentials through ordinary tool output. We call this vulnerability class *filesystem capability laundering*: an out-of-scope filesystem capability is exercised under the guise of an innocuous tool contract.R2a

We examine the gap at three layers. At the *server-category* layer (L1), we compare inferred filesystem capability flags against the server’s functional classification (`primary_type`). Because non-filesystem servers (e.g., memory or code development) often perform legitimate local persistence, category mismatches serve as coarse screening signals. At the *tool-name* layer (L2), a tool’s narrow operation can reach files beyond its intended scope: `git_add` can stage an out-of-repository file whose contents a later diff returns (CVE-2026-27735, §5.2). At the *defense-bypass* layer (L3), an existing path check can be defeated: the Filesystem MCP Server’s string-prefix check admitted sibling directories sharing the allowed prefix (CVE-2025-53110 [30]).R2a, R2b Figure 1 illustrates the staging-and-diff disclosure sequence.R2c

We build MCP-FSD (**F**ilesystem **S**cope **D**etection), a framework that catalogs MCP servers across registries, discovers each server’s tools inside per-repository sandboxes, screens candidates with MCP-aware static filters, and dynamically validates each one against an inferred least-privilege boundary. Across a catalog of 21,097 servers from nine registries, 787 of 4,806 triaged servers (16.4%) receive a filesystem capability flag without a `FileSystem` category label. Dynamic validation yields 150 adjudicated instances under tested policies. We select 13 additional findings for coordinated disclosure and document two Git MCP Server CVEs (§5).

We make three contributions:

- We characterize and formalize how attacker-influenced MCP tool arguments expose filesystem capabilities beyond a tool’s contract and deployment scope.
- We design MCP-FSD to connect source screening and inferred per-tool boundaries to executable evidence, evaluating exploitability, benign operability, and sandbox containment across three complementary dynamic validation cases.

- We measure ecosystem-scale screening signals, confirm 150 vulnerable instances through dynamic validation under tested sandbox policies, and present Git case studies showing how repository operations expose host credentials.

We evaluate these contributions through five research questions (§4). **RQ1**: how frequent are screening signals, and what evidence supports them? **RQ2**: how many candidates survive three-case dynamic validation? **RQ3**: what were the outcomes of coordinated disclosure? **RQ4**: how do violations distribute over language, registry, and tool category? **RQ5**: how often does a confirmed instance need a format-wrapped payload rather than a bare traversal string?R1e

2 Background and Threat Model

2.1 Threat Model

We study local stdio MCP servers [26] launched as separate processes. In typical local deployments, the server runs with the user’s ambient filesystem permissions, and the client’s tool-approval prompt enforces no per-tool path restrictions on the underlying process [6,15,31]. Process isolation alone does not restrict access to the host filesystem unless explicit sandbox policies are applied. Figure 1 traces the resulting dataflow, from the content that carries the adversary’s argument to the file bytes that leave in the response.R2c

The adversary influences the value of an argument that drives a side effect, through content the agent ingests as untrusted input [16,44,34]. The agent supplies it to a tool whose contract appears benign, and the spawned server exercises a host capability outside what that contract implies. We measure filesystem disclosure: an attack succeeds when the MCP response contains the contents of a file outside the tool’s legitimate scope. The evaluation begins at tool invocation and assumes that the agent issues the tested call or sequence and that the server has host permissions to access the target file. Arbitrary server-code modification and compromise of the OS sandbox are outside this model.

2.2 Filesystem Capability Laundering

An MCP tool exhibits *filesystem capability laundering* when its declared contract does not cover the filesystem effect its arguments can steer. Three conditions characterize the mechanism: (i) an implemented effect exceeds the scope justified by the tool contract and deployment, (ii) attacker-influenced arguments select the filesystem target, and (iii) the call is admitted while the surrounding controls fail to constrain that effect.

The three layers describe different aspects of this mechanism: L1 supplies candidates through category/capability screening, L2 identifies the per-tool contract/effect gap, and L3 identifies a defeated implementation control. They are not mutually exclusive: a missing or bypassable path check can implement a per-tool scope violation.R2b

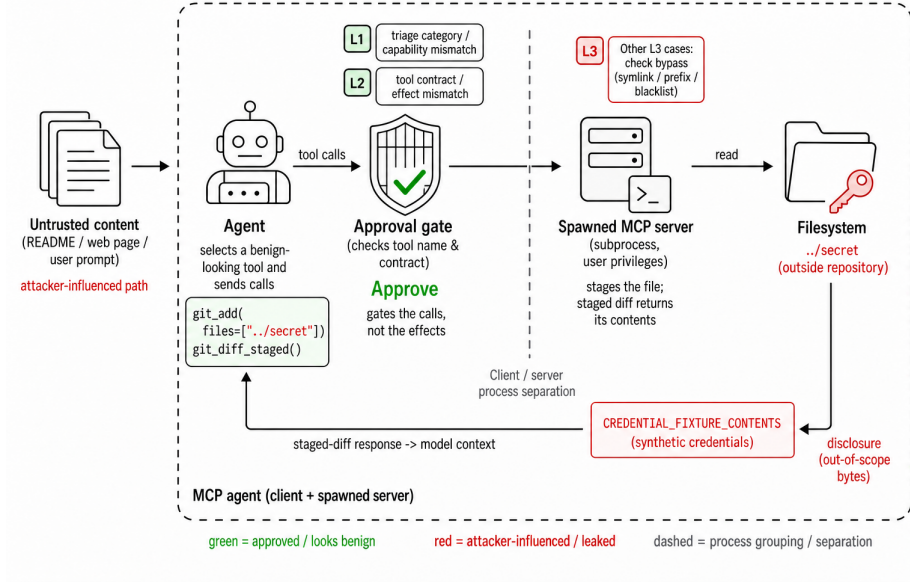


Fig. 1. Disclosure through a staging-and-diff sequence in the studied local-server deployment (§5.2). The repository path is fixed for both calls, while the attacker-influenced file path selects a fixture outside it. Tool approval admits the calls, while the server’s filesystem permissions allow the excess read. The response follows the second call, not staging alone. R2b, R2c

Prompt injection can influence the argument passed to a tool by affecting the model’s instruction-following [24,25]. In contrast, capability laundering centers on the resulting server-side effect. Missing validation or flawed path checks (e.g., path traversal or symlink resolution) provide the underlying implementation mechanisms [1,22].

2.3 Problem Formalization

All effects and responses below refer to a fixed deployment context d , specified by the server version, launch configuration, intended workspace, and initial filesystem state. Let s be an MCP server and $T(s)$ its tools. Invoking a tool $t \in T(s)$ with an argument vector a produces a set of filesystem effects and returns a response $R(t, a)$ to the client:

$$E(t, a) \subseteq \mathcal{F} \times \{\text{read}, \text{write}\},$$

Here $\mathcal{F}$ denotes filesystem targets resolved at access time. Let $\mathcal{A}(t)$ denote the argument vectors an adversary can steer through content the agent ingests (§2.1), and let $\text{Reach}(t) = \bigcup_{a \in \mathcal{A}(t)} E(t, a)$ collect everything those arguments can touch.

Let $C(t) = (\text{name}, \text{description}, \text{schema})$ be the declared contract, and let $B(t; d) \subseteq \mathcal{F} \times \{\text{read}, \text{write}\}$ denote the permissions justified by this contract

and deployment context, including documented roots and legitimate runtime requirements. This *implied scope* differs between a generic file tool authorized for arbitrary paths and a repository tool restricted to a selected workspace.

Definition 1 (Filesystem capability laundering). A tool t launders filesystem capability when $\text{Reach}(t) \not\subseteq B(t; d)$.

Definition 2 (Disclosure witness). A pair (a^*, f^*) witnesses laundering in t when $(f^*, \text{read}) \in E(t, a^*) \setminus B(t; d)$ and the contents of f^* appear verbatim in $R(t, a^*)$. For an explicitly stated call sequence π , we use combined effects $E(\pi)$ and response transcript $R(\pi)$, as in staging followed by a diff.

MCP-FSD computes a policy hypothesis $\hat{B}(t; d)$ from read-write and read-only path sets $(\hat{A}_{rw}, \hat{A}_{ro})$ (Algorithm 1). Reads are allowed in their union and writes in $\hat{A}_{rw}$. A fixture outside this policy tests the witness predicate with $\hat{B}$ substituted for B . We discuss the empirical characteristics and limitations of this inferred boundary in §6.2.R1a, R2d, R3b

Evidence for the intended boundary. Grounding an intended boundary $B(t; d)$ relies on a hierarchy of evidence. Explicit launch configurations, documented workspace roots, and developer-intended containment checks provide strong boundary signals. Intermediate signals, such as base-directory constants, indicate intended storage roots without enforcing permissions. Conversely, heuristic cues such as tool names or subprocess working directories provide weak intent and do not confine access. For each finding, the contract and deployment context must establish why the accessed target lies outside legitimate tool operation (§5.3).R2d, R3b

2.4 Problem Statement and Challenges

Five challenges guide the design of MCP-FSD (§3).

C1: Large, heterogeneous ecosystem. MCP servers are distributed across registries, languages, and packaging conventions. Tool discovery requires source collection, language-specific installation, and transport handling before candidates can enter a common validation pipeline.

C2: Implicit filesystem boundaries. A path parameter indicates where an argument can point, but does not establish which locations are authorized. MCP-FSD combines the declared contract with deployment evidence, such as a selected repository root, to construct the policy hypothesis used during testing.

C3: Library-mediated accesses. Tool handlers often delegate filesystem operations to third-party libraries. For example, `git_add` passes its file list to GitPython. Source review identifies this call, while dynamic testing determines whether the resulting contents reach an MCP response.

C4: Semantically diverse inputs. A tool may accept a single path, a list of files, or a structured document containing a reference. Payload generation must synthesize schema-valid inputs so execution reaches the underlying filesystem operation.

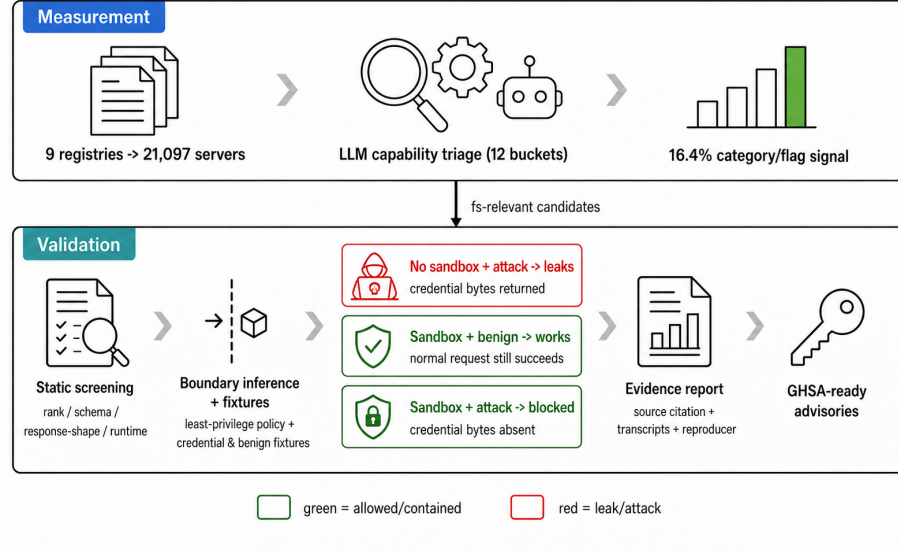


Fig. 2. MCP-FSD as two coupled passes: measurement builds and triages the MCP corpus, while validation applies static screening, boundary inference, generated payloads, and three response-body checks to produce evidence reports.

C5: Connecting filesystem analysis to MCP. Filesystem analysis [20] must identify the callable MCP tools behind each finding and how their effects become observable. MCP-FSD uses tool associations, schemas, response construction, and stdio execution to connect source-level candidates to reproducible calls.

3 MCP-FSD Design

MCP-FSD runs in two passes (Figure 2). The *measurement* pass crawls registries, discovers each server’s tools and schemas, and assigns capability-triage labels. The *validation* pass applies four MCP-aware static filters (§3.1), infers a least-privilege boundary per tool (§3.2), and confirms exploitability with three-case dynamic validation against that boundary (§3.3). The primary disclosure oracle checks for fixture contents in the MCP response. Selected instances use other transcript evidence (§4.3).

3.1 Static Screening

MCP-FSD uses an LLM to identify tool registrations, handlers, and path parameters. It examines how these parameters reach filesystem operations and whether the code validates them. MCP-FSD records each candidate in the Static Analysis Results Interchange Format (SARIF), with its rule, source location, affected tools, parameter, sink, and rationale. We classify each finding into five

rule categories spanning CWE-22 (path traversal) and CWE-59 (link resolution): *PT-001* missing validation, *PT-002* prefix-match containment, *PT-003* URL traversal, *PT-004* symlink TOCTOU (time-of-check-to-time-of-use), and *PT-005* blacklist filter. A small long tail of specialty rules that a language model proposes is reported separately.

The scan reviews the server repository and can flag visible library calls for dynamic testing without reconstructing their effects. It does not exhaustively analyze dependency internals. We call one SARIF result at a reported fix site an *emission*. A finding in a shared helper may affect several tools but produce only one emission. Emissions and dynamically confirmed instances therefore use different counting units. R1d, R2e

Four MCP-aware filters select candidates for dynamic testing based on whether they can plausibly return out-of-scope file contents. *M1* ranks SARIF findings tagged to MCP tools [21]. *M2* reads the tool schema for path-shaped parameters. *M3* inspects response construction for a plausible file-content return. *M4* keeps only candidates that run as stdio MCP servers. These lightweight filters prioritize high-probability candidates for dynamic testing without the overhead of whole-program taint tracking.

For the disclosure-oriented pass, *M1* prioritizes file-content sinks over write-only or shell findings. *M2* identifies the argument that reaches the sink, including entries in a tool’s file list. *M3* distinguishes a response containing file text from one reporting only derived metadata: a line count or schema descriptor cannot satisfy the credential-content oracle. *M4* checks whether the candidate can be exercised in the local stdio harness. It excludes hosted services whose runtime cannot be reproduced.

Worked screening example. For `git_add`, the reviewed handler connects the schema’s `files` parameter to `repo.index.add(files)` without a repository-containment check, recorded as a PT-001 SARIF candidate. *M1* associates it with the MCP tool and *M2* identifies its path input. The disclosure observation comes from `git_diff_staged`, so *M3* uses the explicitly stated staging-and-diff sequence rather than treating a staging acknowledgment as file content. *M4* checks the stdio launch. While this evaluation validates this specific multi-call chain, dynamic validation confirms whether the final response discloses fixture contents. R2e, R2f

3.2 Boundary Inference Algorithm

For each candidate that passes static screening, boundary inference constructs the least-privilege scope used by payload generation and sandbox validation. MCP-FSD derives the scope from the tool’s legitimate operation rather than from the suspected bug: read-write access covers the working directory, while read-only access covers the server binary, dependency tree, and required system-root subtree. The fixture planner then places the credential fixture outside every allow scope and the benign marker inside the working scope.

Algorithm 1 Boundary inference for an MCP tool t .

Require: launch tuple L , schema S , description D , repo signals R

Ensure: scope $(\hat{A}_{rw}, \hat{A}_{ro})$ as glob sets

```
1:  $L_{\text{root}} \leftarrow \text{EXTRACTROOTS}(L)$   $\triangleright$  cwd, -root, env path roots
2:  $S_{\text{path}} \leftarrow \text{PATHPARAMS}(S)$   $\triangleright$  parameters whose type or name encodes a path
3:  $R_{\text{static}} \leftarrow \text{HANDLERPATHREFS}(R)$   $\triangleright$  source-visible paths and modes
4:  $\mathcal{P} \leftarrow \text{LMDRAFTPOLICY}(L_{\text{root}}, S_{\text{path}}, D)$   $\triangleright$  draft candidate scope
5: if  $R_{\text{static}} \not\subseteq \mathcal{P}$  then
6:    $\mathcal{P} \leftarrow \mathcal{P} \cup \text{COVER}(R_{\text{static}})$   $\triangleright$  cover identified runtime needs
7: end if
8: return  $\text{SPLITRWRO}(\mathcal{P}, S)$   $\triangleright$  partition by per-parameter read/write evidence
```

Algorithm. Algorithm 1 uses four signals: launch configuration L , path-bearing schema fields S , the tool description D , and repository source signals R . The language model drafts a candidate policy from (L, S, D) . MCP-FSD extends the policy to cover runtime requirements identified in R , then tests whether a benign invocation succeeds.

Subroutines. **EXTRACTROOTS** reads the launch tuple: the working directory the client starts the server in, any **--root**-style launch argument, and path-valued environment variables. **PATHPARAMS** walks the JSON Schema and returns the fields whose type, name, or format encodes a path, so payload generation targets the appropriate argument (C4). **HANDLERPATHREFS** identifies literal and constructed paths and access modes in the reviewed handler code. **LMDRAFTPOLICY** asks the model for a candidate glob policy from those roots, fields, and the description. **COVER** extends the policy to include identified runtime dependencies (such as language runtimes, libraries, and system configuration paths). **SPLITRWRO** partitions the result into a read-write and a read-only half according to whether each parameter is written or only read at the observed sites.R2f

Checks and example. Before dynamic testing, MCP-FSD checks that the policy covers identified runtime requirements and leaves a non-empty out-of-scope region for the fixture. For **git_add** this yields **/ws/**** read-write and dependencies read-only, with the benign file inside **/ws** and the credential fixture outside. The three-case test then replays staging followed by a diff (§5.2).R2f

The repository needs write access for the index and object-database updates required by staging, while runtime dependencies need only be readable. The credential fixture must lie outside both sets: placing it outside the working tree alone would be insufficient if a read-only runtime allowance still covered it. The benign invocation checks that these permissions support the intended staging-and-diff operation.

3.3 Payload Generation and Dynamic Three-Case Validation

Dynamic payload generation. For each path-shaped parameter, MCP-FSD first fixes two targets from the inferred boundary: the malicious target is the planted

out-of-scope credential fixture, and the benign target is the in-scope marker file. A language model then proposes concrete argument forms, seeded with published path-traversal exemplars, and refines them against the tool schema, the inferred boundary, and observed responses until the malicious attempt reaches the fixed credential target or the attempt budget is exhausted. For a tool that parses structured input, it wraps the same target in a format-valid container, for example a BPMN, glTF, DOCX, or PNG file, or a Terraform body. The model varies the argument form while the target stays fixed. MCP-FSD resets test state and replays the same converged payload or stated call sequence with and without the sandbox.

Response-disclosure validation uses three cases. Other effects require separate evidence rules (§4.3). PoC 1 runs the malicious payload without the sandbox and succeeds only when the response contains the planted credential contents (C2–C3). PoC 2 runs the benign in-scope payload with the sandbox and requires the expected successful operation with no denial event. PoC 3 replays PoC 1’s malicious payload with the sandbox and requires absent credential contents plus an explicit denial or a non-disclosing response.

This three-case protocol isolates exploitability from containment: PoC 1 confirms that the exploit succeeds on an unconfined host; PoC 2 verifies that the inferred sandbox policy preserves legitimate, in-scope functionality; and PoC 3 demonstrates that the sandbox neutralizes the exploit under that exact policy. Together, these cases establish that detected disclosures represent genuine vulnerabilities remediated by least-privilege containment.R1c

We distinguish explicit containment, where the response exposes the operating-system denial, from silent containment, where the response is empty or non-disclosing without an exposed denial. A silent response alone does not identify which syscall was blocked. Throughout, we call this enforcement *operating-system process isolation*, reserving “kernel-enforced” for backends that provide kernel-level enforcement such as Landlock or Seatbelt [2,7,18]. All 150 dynamically adjudicated instances in §4.3 pass this three-case gate, providing the containment evidence discussed in §6.1.R1c

Per candidate, MCP-FSD records the three parsed response transcripts and the reproducer script, along with source citations, the inferred sandbox policy, runtime details and, when available, a sink citation pinned to a specific file and line. The verdict adjudicator reads this per-candidate record.

4 Evaluation

4.1 Experiment Setup

Data collection. We catalog 21,097 listings from nine registries, including 17,919 open-source entries. The measurement pass selects local open-source servers, excluding forks, archived repositories, remote-hosted services, and unavailable code. Successful startup and discovery produce the triage input. The validation pass then screens and tests selected candidates.

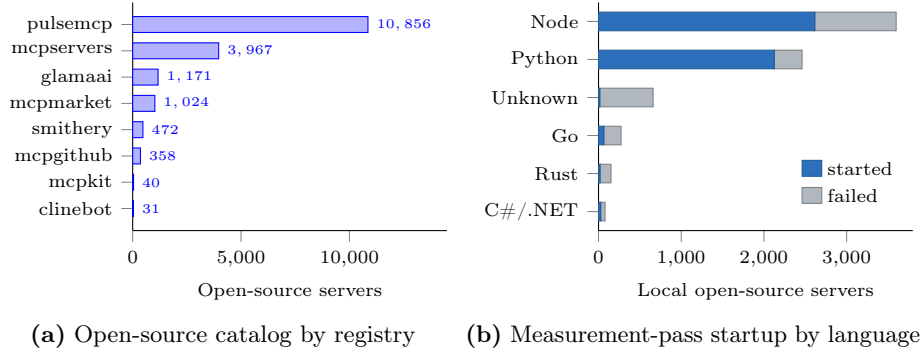


Fig. 3. Corpus composition: (a) 17,919 open-source listings by registry. (b) Startup outcomes for 7,222 local open-source servers, by language: 86.3% of Python and 72.7% of Node servers start, against 34.2% (C#/.NET), 23.5% (Go), and 11.9% (Rust).

The triage step classifies each server across twelve functional categories (**primary_type**), such as FileSystem, CodeDev, and KnowledgeRetrieval, derived via LLM analysis of tool descriptions rather than external registry tags.R2a

Implementation and corpus funnel. MCP-FSD comprises 7,486 executable lines of Python and shell plus 618 lines of analysis prompts. Dynamic validation runs each candidate in **nono** v0.43.0 [2], a macOS Seatbelt sandbox. Static analysis returns results for 4,292 servers and flags 2,818 of them for potential vulnerabilities (20.3% and 13.4% of the 21,097-server catalog, respectively).

Units of analysis. Our evaluation distinguishes four units of analysis: *servers* (catalog and triage entries); *static emissions* (individual SARIF finding sites, where a shared helper may affect multiple tools); *dynamic instances* (unique tool-policy pairs confirmed via three-case replay); and *disclosure packages* (curated multi-tool findings bundled for vendor remediation).

Triage snapshot and selection bias. The final triage snapshot contains 4,806 servers, with static analysis evaluated on the successfully cloned and parsed subset. The funnel is selective: only 7,222 of 21,097 catalogued servers (34.2%) reach the measurement pass, the post-measurement corpus is 97% Node and Python, and pulsemcp supplies 84% of measurement-pass startups. Our prevalence estimates and per-registry comparisons describe this locally runnable open-source corpus rather than an idealized, uniform distribution across all ecosystems.

4.2 RQ1: Screening Signals and Evidence

The L1 signal combines the triage’s multi-label **primary_type** field and four filesystem capability flags (file-related call, path input, file read, and file write), extracted via LLM analysis of tool discovery summaries. Among 4,806 servers, 787

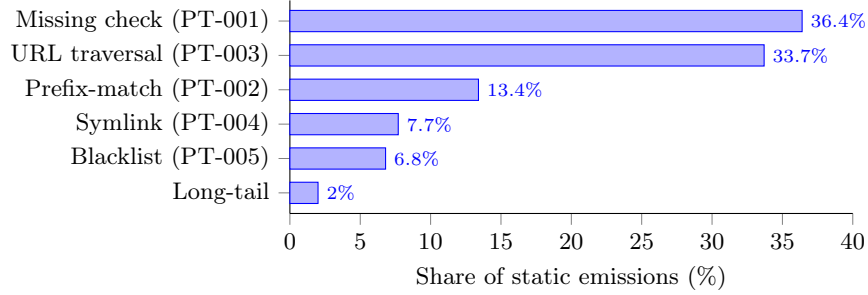


Fig. 4. Failure-mode distribution over 10,304 static emissions from 2,818 flagged servers.

(16.4%) exhibit at least one filesystem capability flag despite lacking a FileSystem category classification. Because routine persistence mechanisms do not necessarily violate a tool’s contract, and absent external ground-truth labels across the entire catalog, this category mismatch serves as a coarse initial screening signal rather than a definitive violation metric. R3a, R3b

At the static-emission level, MCP-FSD emits 10,304 findings across the 2,818 flagged servers (65.7% of the 4,292 with results), with missing path validation as the largest individual rule category (Figure 4). Combined, the defense-bypass rules (PT-002–PT-005, 61.6%) outweigh PT-001 missing-validation (36.4%). Rule families rank differently when counted by confirmed instances rather than static emissions: a single repository can emit many findings across sibling tools, increasing the weight of whichever rule it carries. Each emission has a severity level: 61.8% are reported at **error** level, 29.8% at **warning**, and 8.4% at **note**.

URL-traversal emissions include candidates that do not disclose local files, so static counts remain separate from dynamic outcomes.

Layer-classifier accuracy. Because the L2 and L3 results rest on automated classification, we validate each classifier against a hand-labeled sample of about 300 candidates (Table 1). The L2 tool-name classifier reaches 99.5% precision and 97.1% recall over 300 labeled tools (one false positive, six false negatives). The L3 defense-bypass classifier achieves 95.3% precision and 99.3% recall over 297 labeled findings (seven false positives, one false negative). The remaining three repositories in the 300-sample set were excluded because they were deleted, made private, or inaccessible at validation time. These benchmarks measure static classification precision; end-to-end authorization fidelity is evaluated dynamically in RQ2.R3a

4.3 RQ2: Confirmed Instances

MCP-FSD confirms 150 instances, each adjudicated under the three-case dynamic validation (§3.3) against the inferred least-privilege boundary. For 136 of the 150 instances, adjudication relies on exact credential-marker matching: PoC 1’s response contains the planted secret string, whereas PoC 3’s response contains

Table 1. Per-layer detection accuracy. Pos./Neg. are ground-truth positive/negative counts on the hand-labeled, stratified benchmark for each layer (300 for L2, 297 for L3). L1 accuracy is unmeasured because no independent labeled benchmark exists.

Layer	Pos.	Neg.	TP	FP	FN	Prec.	Rec.	F1
L1 server-category	–	–	–	–	–	–	–	–
L2 tool-name	205	95	199	1	6	0.995	0.971	0.983
L3 defense-bypass	142	155	141	7	1	0.953	0.993	0.972
Micro	347	250	340	8	7	0.977	0.980	0.978
Macro	–	–	–	–	–	0.974	0.982	0.978

no marker bytes. For the remaining 14, the adjudicator evaluates corresponding evidence from other transcript features (write side-effects, response-shape changes, or external-CLI outputs).R1c, R3b

4.4 RQ3: Disclosed Outcomes

We reported 13 findings, including two CVEs in Anthropic’s Git MCP Server (§5). MCP-FSD dynamically validates `git_add` with a full three-case PoC. The `git_init` finding is static because its disclosure is indirect: it seeds a repository that downstream Git tools then read. Both CVEs have been patched upstream.

Among the 13 packages, 11 cite source-level mechanisms or configuration as evidence, including validators, storage-base constants, and shell argument handling. These evidence sources span a spectrum of enforcement: a base directory may lack runtime checks, while an overbroad allowlist or shell-quoting flaw exhibits a distinct authorization boundary from a containment bypass. Two packages rely on contract-implied scope boundaries without explicit path validation at the read sink. All 13 packages were submitted for coordinated disclosure, with upstream fixes or CVE assignments pending at the reporting cutoff.R2d, R3b, R2h

We prioritized findings where either a source-level control is bypassed or the project publicly commits to a narrow scope that the implementation does not enforce, because those reports have a per-server fix without requiring a protocol change.

4.5 RQ4: Violation Distribution

The flagged population follows the same concentration as the corpus: pulsemcp dominates the registry axis, and Node/Python dominate the language axis. At the sink level, 54.3% of emissions are Node-ecosystem files and 42.6% are Python files. All other languages together account for under 3.1%. Python leads PT-001 missing-validation (53.8% within-rule share), while Node leads every bypass-style rule at 57%–63% (Figure 5a). This distribution reflects structural tooling gaps: the MCP TypeScript SDK provides no built-in path-containment primitive (such as a `safePath()` utility), leaving Node servers prone to flawed

Table 2. Filesystem footprint in the final triage snapshot. The L1 row counts a positive filesystem flag without a FileSystem label.

Measurement	Count	Share of triaged
Triaged servers	4,806	100.0%
FileSystem label	1,345	28.0%
Any filesystem capability flag	2,131	44.3%
L1 screening signal	787	16.4%

custom checks (accounting for 54.3% of all emissions). Conversely, Python servers frequently omit validation checks entirely (53.8% of PT-001). These patterns indicate distinct remediation paths: default allowlist helpers for Python and standardized path-containment primitives for TypeScript.

Table 2 summarizes the filesystem footprint across the 4,806 triaged servers. In total, 787 servers exhibit filesystem capability flags without a FileSystem category classification, forming the L1 screening candidate set.

4.6 RQ5: Payload Forms

We categorize the payloads synthesized by the iterative generation loop (§3.3) by their converged input format (bare traversal paths versus format-valid containers). Across the 150 confirmed instances, 144 (96.0%) use the bare traversal form, while 6 (4.0%) require a format-valid container. Among the 38 confirmed instances that bypass an explicit source-level validation check, 32 (84.2%) use the bare form and the same 6 (15.8%) require wrapping.

Format-valid fixtures ensure that tools parsing structured inputs (such as glTF, DOCX, or Terraform configurations) accept the payload before executing underlying filesystem calls. An unformatted traversal string rejected by an input parser would leave the tool’s access to the target untested. While 96.0% of vulnerable tools accept bare paths, structured tools necessitate format-valid containers to expose the laundering path.

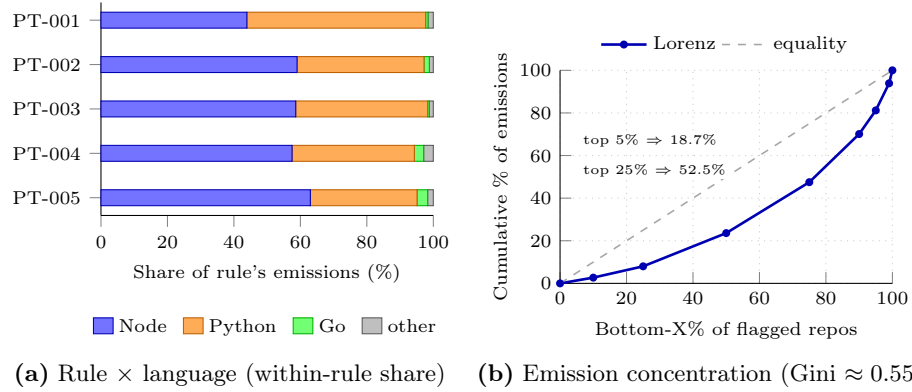


Fig. 5. Static findings by language and repository. (a) Python contributes the largest share of PT-001 emissions, while Node leads PT-002–PT-005. PT-003 subsumes URL-based access patterns. (b) Across 2,818 flagged repositories, the top 5% of repos produce 18.7% of emissions and the top 25% produce 52.5%.

5 Case Studies

Two CVEs in Anthropic’s Git MCP Server illustrate L2 laundering across distinct boundaries. MCP-FSD flags both and dynamically validates `git_add` (§4.4).

5.1 Case 1 (L2): Unscoped Repository Creation in `git_init` (CVE-2025-68143)

The `git_init` handler initializes a repository at the supplied `repo_path`:

```
# git_init(repo_path)
repo_path = Path(arguments["repo_path"])
repo = git.Repo.init(path=repo_path, mkdir=True)
```

Root cause. The handler passes the caller-supplied `repo_path` directly to `Repo.init()`, without verifying containment within the configured workspace. Any writable filesystem location becomes a valid initialization target. **Impact.** When an attacker-influenced argument names a sensitive directory such as `~/ .ssh`, `git_init` converts that directory into a Git repository. Subsequent Git tool calls such as `git_add` and `git_diff_staged` then operate inside it legitimately and return raw file contents, including SSH private keys, as diff output to the agent. In this deployment, the selected workspace defines the intended scope for repository creation. The issue was assigned CVE-2025-68143 [13]. The fix removes `git_init` from the server (released as version 2025.9.25). The upstream advisory credits Yarden Porat for the initial report and one of our co-authors (@Odd) for developing the upstream patch. We contributed independent vulnerability analysis and remediation.

Under this workspace boundary, the vulnerability satisfies all three capability-laundering conditions (§2.2): repository creation exceeds the authorized workspace scope (i), caller-supplied arguments steer the filesystem target (ii), and the server executes the initialization without containment (iii), enabling subsequent credential extraction.

5.2 Case 2 (L2): Path-Traversal Staging in `git_add` (CVE-2026-27735)

The `git_add` handler stages the supplied `files` list:

```
# git_add(repo_path, files=[])
repo = git.Repo(repo_path)
repo.index.add(files)
```

Root cause. The handler does not check whether paths in `files` stay within the repository root before staging. The affected `_to_relative_path()` function in GitPython returns relative paths without validating their containment. Paths containing `../` then reach `_store_path()`, which reads the file bytes into Git’s object database. The contents are stored by Git even though the source file lies outside the working tree. The fix, released in 2026.1.14, delegates staging to the Git CLI through `repo.git.add()`. The CLI rejects out-of-repository paths [14]. **Impact.** Sensitive host files such as SSH keys or dotfiles can be staged and then exposed through a staged diff, or carried into a commit that sends them to a remote on the next push. The repository root forms the implied scope of `git_add`; failing to enforce containment allows out-of-scope files to be staged into the repository index. The issue was assigned CVE-2026-27735 [14]. The complete three-case dynamic validation runs against the pre-patch version: the unsandboxed staging-and-diff sequence returns the planted credential contents, the sandboxed benign sequence succeeds on an in-scope file, and the sandboxed malicious sequence returns a permission-denied error (`EPERM`) with the credential bytes absent.

5.3 Synthesis

The cases require containment at different points in the call sequence. For `git_init`, the creation target must lie within the deployment’s workspace before a repository is created there. Once a sensitive directory has become a repository, later calls that name the same `repo_path` can stage and display its files without leaving that repository root. A check against only the newly selected root would therefore miss the earlier workspace violation. For `git_add`, the repository already exists: each entry in `files` must remain within it. The preceding state-changing call makes the bytes reachable, and the staged-diff response reveals the disclosure.

In both cases, the intended scope, caller-selected path, and missing containment instantiate conditions (i)–(iii) of §2.2. Neither case requires a pre-existing containment check. The same reasoning applies to other tools: a configured vault root supports an intended scope, whereas a suggestive name such as `read_build_log` exhibits greater scope ambiguity. R2d, R3b

Prior L3 vulnerabilities in the reference Filesystem MCP Server—such as sibling-prefix bypass (CVE-2025-53110 [30]) and symlink traversal (CVE-2025-53109 [29])—provide external benchmark cases. MCP-FSD flags their source-level patterns to verify static rule coverage, though they remain separate from our 13 newly reported findings.R2h

6 Discussion

6.1 Mitigations

At L3, developers should enforce canonical-path containment, validate schema fields, and address symlink races at access time. An SDK-level containment helper would reach the many Node servers that currently ship no such check (§4.5). At L2, a protocol extension could expose a per-tool `fs_scope` field in `tools/list`, allowing clients to inspect the requested read/write scope before approval and enforce the approved policy on server execution [40]. MCP-FSD’s read-write and read-only path sets offer a candidate representation for that field. Inferred policies would need review against deployment requirements. Our three-case validation demonstrates that operating-system process isolation reliably enforces least-privilege boundaries without disrupting benign tool functionality. At L1, registry screening can identify filesystem capabilities for further review.

6.2 Limitations

First, the pipeline only validates servers it can install and run. The dynamic results are conditional on successful startup and the selected runtime configurations. Second, dynamic validation may miss a violation whose out-of-scope effect never appears in a response, or that requires a runtime precondition the harness does not reproduce. Third, we do not measure the accuracy of the inferred boundaries themselves. A policy that is too restrictive can flag legitimate access, while one that is too permissive can miss a violation. Source coverage and a successful benign call do not establish authorization accuracy. Future benchmark efforts should establish expert-annotated boundaries across diverse server documentation to systematically quantify boundary inference precision and recall.R3b

Fourth, the effect model includes writes, but the disclosure-oriented evaluation does not systematically measure unauthorized creation, modification, or deletion. The `git_init` case illustrates write-driven setup, and our separate Memory MCP Server report demonstrates configuration injection through a persistence write when an attacker controls the launch-time persistence path. Its fix was merged upstream without a CVE.R2g

Assessing these writes requires filesystem-state evidence as well as responses. In a targeted case study on `mcp-architect`, a caller-controlled `projectId` is joined to the storage root without containment: an unsandboxed invocation successfully writes a sentinel file outside the allowed storage directory, whereas sandboxed replay is blocked by the OS policy (`EPERM`). This demonstrates how

an integrity-focused oracle can detect unauthorized filesystem writes even when no file contents are returned in tool responses.R2g

Fifth, enforcement strength follows the sandbox backend (§3.3). Sixth, validation covers selected tools and explicitly stated call sequences, not systematic discovery of cross-tool chains. A tool that writes attacker-controlled content into a file that a second tool later reads can launder capability across the pair while each tool, evaluated alone, respects its boundary.

Seventh, several stages rely on LLM-generated judgments, namely the capability-triage labels used in the L1 signal (§4.2), the rule families behind the static emissions, and the boundary-policy drafts. Those results inherit the model’s error modes. A systematic accuracy study of these model-dependent stages remains future work, along with full reporting of model versions, prompts, and decoding settings.

6.3 Ethical Considerations

This work analyzes publicly available MCP servers and validates disclosure against fixture credentials, never real secrets. We follow responsible-disclosure practice: the Git MCP cases of §5 were reported to the affected vendor, and both received CVE assignments. Dynamic validation runs on controlled installations with synthetic fixtures and resets between cases. PoC 1 intentionally omits the test sandbox.

7 Related Work

MCP security and filesystem detection. Table 3 situates MCP-FSD among MCP security studies [41,38,36,37,35,17] and filesystem-flaw detectors [1,22,20,21]. The closest contract-analysis work, DCIChecker [35], combines structured code extraction with direct, reverse, and arbitration prompts to detect description-code inconsistencies. MCP-FSD focuses on filesystem effects triggered by attacker-controlled arguments and tests them against an inferred policy for each tool.R1b

While MCP-SandboxScan [37] utilizes WASI sandboxing and output matching to detect external-input leakage, MCP-FSD couples static screening with dynamic three-case replay on native host runtimes, capturing complex library-mediated flows against inferred per-tool scopes. Similarly, where mobile analyzers such as PathSentinel [20] evaluate applications against pre-existing platform access-control policies, MCP-FSD infers least-privilege boundaries directly from unstandardized tool descriptions and deployment configurations.R1b

Description-code agreement, file-to-output flow, and authorization answer different questions. A description can promise to stage local files without specifying which directories are allowed. A diff response establishes that file bytes reached the client, while the repository-root policy states the scope against which the access is tested. MCP-FSD records the source candidate, policy, and execution transcripts together, connecting the declared operation to the tested access and returned bytes.

Table 3. Related work by analysis target and validation evidence.

Work	Target / scope	Evidence / method
PathSentinel [20]	Android filesystem attacks and platform access-control policy	Program and policy analysis with generated exploit validation
MICRYSCOPE [41]	Cryptographic misuse in MCP servers	Cross-language API and dependency analysis with taint rules
MCPGuard [38]	MCP threats and defenses	Systematization of scanning and runtime-monitoring approaches
MCP-SandboxScan [37]	External-input exposure and filesystem capability violations	WASM/WASI execution with output matching and denial evidence
DCIChecker [35]	Description-code inconsistency, including undeclared effects	Structured code extraction and LLM cross-validation
MCP-FSD	Filesystem effects relative to per-tool policy hypotheses	Source screening and fixture-based three-case execution under OS containment

Agent security, prompt injection, and isolation. Related work benchmarks harmful tool use [33,10,3,45,43,23], studies agent backdoors [39,42], and attacks or defends model instruction-following [9,32,46,11]. It also applies models to offensive-security tasks [12,19] and documents path traversal in the agent harness itself [5,4]. MCP-FSD tests the effects of separately launched MCP server processes using existing OS containment and deception primitives [18,40,8,7,2]. Paired malicious replay and a benign control test the inferred policy (§3.3).

8 Conclusion

Filesystem capability laundering exposes a fundamental gap between client-side tool approval and unconfined server filesystem execution. MCP-FSD bridges this gap by coupling static screening, inferred least-privilege boundaries, and dynamic three-case validation across 21,097 cataloged servers. Real-world vulnerabilities in foundational tools—such as repository creation and staging flaws in Anthropic’s Git server—demonstrate how ordinary tool interactions can inadvertently leak sensitive host credentials. These findings demonstrate the necessity of ecosystem-scale registry screening and explicit, runtime-enforced per-tool filesystem contracts.

References

1. Akhoundali, J., Hamidi, H., Rietveld, K., Gadyatskaya, O.: Eradicating the unseen: Detecting, exploiting, and remediating a path traversal vulnerability across GitHub.

- In: Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS) (2025). <https://doi.org/10.1145/3708821.3736220>
2. always-further: nono: Capability-based sandbox using seatbelt and landlock. <https://github.com/always-further/nono> (2025), accessed: 2026-06-21
 3. Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Gal, Y., Davies, X.: AgentHarm: A benchmark for measuring harmfulness of LLM agents. In: International Conference on Learning Representations (ICLR) (2025)
 4. Anthropic: Claude Code path-traversal security advisory (GHSA-66m2-gx93-v996). <https://github.com/anthropics/claude-code/security/advisories/GHSA-66m2-gx93-v996> (2026), accessed: 2026-06-21
 5. Anthropic: Claude Code path-traversal security advisory (GHSA-pmw4-pwvc-3hx2). <https://github.com/anthropics/claude-code/security/advisories/GHSA-pmw4-pwvc-3hx2> (2026), accessed: 2026-06-21
 6. Anthropic: Connect Claude Code to tools via MCP. <https://code.claude.com/docs/en/mcp> (2026), accessed: 2026-06-21
 7. Anthropic Experimental: Anthropic sandbox runtime (srt). <https://github.com/anthropic-experimental/sandbox-runtime> (2025), accessed: 2026-06-21
 8. Ayzenshteyn, D., Weiss, R., Mirsky, Y.: Cloak, honey, trap: Proactive defenses against LLM agents. In: Proceedings of the USENIX Security Symposium (2025)
 9. Chen, S., Piet, J., Sitawarin, C., Wagner, D.: StruQ: Defending against prompt injection with structured queries. In: Proceedings of the USENIX Security Symposium (2025)
 10. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramer, F.: AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track (2024)
 11. Deng, G., Liu, Y., Li, Y., Wang, K., Zhang, Y., Li, Z., Wang, H., Zhang, T., Liu, Y.: MASTERKEY: Automated jailbreaking of large language model chatbots. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2024). <https://doi.org/10.14722/ndss.2024.24188>
 12. Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Liu, Y.: PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In: Proceedings of the USENIX Security Symposium (2024)
 13. GitHub Advisory Database: CVE-2025-68143: Unscoped repository creation in the Anthropic Git MCP Server (GHSA-5cgr-j3jf-jw3v). <https://github.com/advisories/GHSA-5cgr-j3jf-jw3v> (2025), accessed: 2026-06-21
 14. GitHub Advisory Database: CVE-2026-27735: Path-traversal staging in the Anthropic Git MCP Server (GHSA-vjqx-cfc4-9h6v). <https://github.com/advisories/GHSA-vjqx-cfc4-9h6v> (2026), accessed: 2026-06-21
 15. Google: Gemini CLI: MCP server. <https://github.com/google-gemini/gemini-cli/blob/main/docs/tools/mcp-server.md> (2026), accessed: 2026-06-21
 16. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec) (2023). <https://doi.org/10.1145/3605764.3623985>
 17. Hou, X., Zhao, Y., Wang, S., Wang, H.: Model Context Protocol (MCP): Landscape, security threats, and future research directions. arXiv preprint arXiv:2503.23278 (2025), <https://arxiv.org/abs/2503.23278>

18. Jia, J., Zhu, Y., Williams, D., Arcangeli, A., Canella, C., Franke, H., Feldman-Fitzthum, T., Skarlatos, D., Gruss, D., Xu, T.: Programmable system call security with eBPF. arXiv preprint arXiv:2302.10366 (2023), <https://arxiv.org/abs/2302.10366>
19. Kim, H., Song, M., Na, S.H., Shin, S., Lee, K.: When LLMs go online: The emerging threat of web-enabled LLMs. In: Proceedings of the USENIX Security Symposium (2025)
20. Lee, Y.T., Vijayakumar, H., Qian, Z., Jaeger, T.: Static detection of filesystem vulnerabilities in Android systems. arXiv preprint arXiv:2407.11279 (2024)
21. Lekssays, A., Mouhcine, H., Tran, K., Yu, T., Khalil, I.: LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models. In: Proceedings of the USENIX Security Symposium (2025)
22. Lin, E., Koishybayev, I., Dunlap, T., Enck, W., Kapravelos, A.: UntrustIDE: Exploiting weaknesses in VS Code extensions. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2024). <https://doi.org/10.14722/ndss.2024.24073>
23. Liu, F., Zhang, Y., Luo, J., Dai, J., Chen, T., Yuan, L., Yu, Z., Shi, Y., Li, K., Zhou, C., Chen, H., Yang, M.: Make agent defeat agent: Automatic detection of taint-style vulnerabilities in LLM-based agents. In: Proceedings of the USENIX Security Symposium (2025)
24. Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., Zhang, L.Y., Liu, Y.: Prompt injection attack against LLM-integrated applications. arXiv preprint arXiv:2306.05499 (2023)
25. Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z.: Formalizing and benchmarking prompt injection attacks and defenses. In: Proceedings of the USENIX Security Symposium (2024)
26. Model Context Protocol: Model Context Protocol specification: Transports. <https://modelcontextprotocol.io/docs/concepts/transports> (2025), accessed: 2026-06-21
27. Model Context Protocol: Specification 2025-11-25: Roots. <https://modelcontextprotocol.io/specification/2025-11-25/client/roots> (2025), accessed: 2026-06-21
28. Model Context Protocol: Specification 2025-11-25: Tools. <https://modelcontextprotocol.io/specification/2025-11-25/server/tools> (2025), accessed: 2026-06-21
29. NIST National Vulnerability Database: CVE-2025-53109: Symlink boundary bypass in the Anthropic Filesystem MCP Server. <https://nvd.nist.gov/vuln/detail/CVE-2025-53109> (2025), accessed: 2026-06-21
30. NIST National Vulnerability Database: CVE-2025-53110: Prefix-match containment bypass in the Anthropic Filesystem MCP Server. <https://nvd.nist.gov/vuln/detail/CVE-2025-53110> (2025), accessed: 2026-06-21
31. OpenAI: Codex CLI: Sandboxing. <https://developers.openai.com/codex/concepts/sandboxing> (2026), accessed: 2026-06-21
32. Piet, J., Alrashed, M., Sitawarin, C., Chen, S., Wei, Z., Sun, E., Alomair, B., Wagner, D.: Jatmo: Prompt injection defense by task-specific finetuning. arXiv preprint arXiv:2312.17673 (2024)
33. Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C.J., Hashimoto, T.: Identifying the risks of LM agents with an LM-emulated sandbox. In: International Conference on Learning Representations (ICLR) (2024)
34. Shi, J., Yuan, Z., Tie, G., Zhou, P., Gong, N.Z., Sun, L.: Prompt injection attack to tool selection in LLM agents. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2026)

35. Shi, Y., Zhang, X., Zhang, X., Shen, X., Ouyang, H., Qiu, H., Zhang, M., Yang, M.: Description-code inconsistency in real-world MCP servers: Measurement, detection, and security implications. arXiv preprint arXiv:2606.04769 (2026), <https://arxiv.org/abs/2606.04769>
36. Siameh, T., Addobe, A.A., Liu, C.H.: Context injection vulnerabilities and resource exploitation attacks in Model Context Protocol. TechRxiv preprint (2025). <https://doi.org/10.36227/techrxiv.175321790.02502754/v1>
37. Tan, Z., Hao, R., Singer, J., Tang, Y., Anagnostopoulos, C.: MCP-SandboxScan: WASM-based secure execution and runtime analysis for MCP tools. arXiv preprint arXiv:2601.01241 (2026), <https://arxiv.org/abs/2601.01241>
38. Wang, B., Liu, Z., Yu, H., Yang, A., Huang, Y., Guo, J., Cheng, H., Li, H., Wu, H.: MCPGuard: Automatically detecting vulnerabilities in MCP servers. arXiv preprint arXiv:2510.23673 (2025), <https://arxiv.org/abs/2510.23673>
39. Wang, Y., Xue, D., Zhang, S., Qian, S.: BadAgent: Inserting and activating backdoor attacks in LLM agents. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL) (2024). <https://doi.org/10.18653/v1/2024.acl-long.530>
40. Wu, Y., Roesner, F., Kohno, T., Zhang, N., Iqbal, U.: IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2025)
41. Yan, B., Zhang, Y., Xu, M., Wu, H., Zhang, Y., Li, K., Zhang, G., Cheng, X.: MCP does not stand for misuse cryptography protocol: Uncovering cryptographic misuse in Model Context Protocol at scale. arXiv preprint arXiv:2512.03775 (2025), <https://arxiv.org/abs/2512.03775>
42. Yang, W., Bi, X., Lin, Y., Chen, S., Zhou, J., Sun, X.: Watch out for your agents! investigating backdoor threats to LLM-based agents. In: Advances in Neural Information Processing Systems (NeurIPS) (2024)
43. Yuan, T., He, Z., Dong, L., Wang, Y., Zhao, R., Xia, T., Xu, L., Zhou, B., Li, F., Zhang, Z., Wang, R., Liu, G.: R-Judge: Benchmarking safety risk awareness for LLM agents. In: Findings of the Association for Computational Linguistics (EMNLP Findings) (2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.79>
44. Zhan, Q., Liang, Z., Ying, Z., Kang, D.: InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of the Association for Computational Linguistics (ACL Findings) (2024). <https://doi.org/10.18653/v1/2024.findings-acl.624>
45. Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., Zhang, Y.: Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In: International Conference on Learning Representations (ICLR) (2025)
46. Zhong, Y., Miao, Q., Chen, Y., Deng, J., Cheng, Y., Xu, W.: Attention is all you need to defend against indirect prompt injection attacks in LLMs. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2026)